EMBEDDED ONBOARDING · CONTROL TEAM

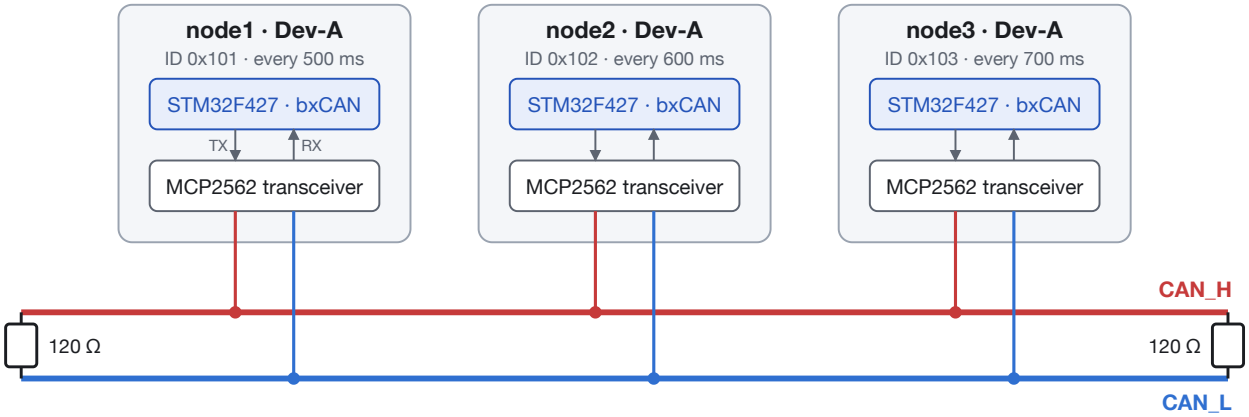
CAN Bus Basics

Three simulated RoboMaster Dev-A boards talking over one CAN bus. You write real STM32 HAL code and run it in the Renode simulator. No hardware is needed until chapter 8.

TIME
About 2 hours

YOU NEED
A GitHub account (for Codespaces) or a computer with Renode and the ARM compiler

YOU'LL LEARN
CAN frames, IDs, filters, bit timing, and how to send and receive with the STM32 HAL



What you'll build. Three boards run the same firmware and share one CAN bus. In the simulator, the wires, transceivers and resistors are replaced by a single virtual “CAN hub”.

1 What you'll build
2 CAN in ten minutes
3 Set up
4 Build and run the simulation
5 Walk through the code
6 Exercises
7 Check your work automatically
8 From the simulator to real boards
9 Troubleshooting
A Solutions
B DJI motor CAN frames
C Renode cheat sheet

1 What you'll build

On the robot, the main controller talks to the motors, the gimbal and the other boards over **CAN**. Before you touch a real robot, you'll learn CAN on three *simulated* RoboMaster Development Board Type A ("Dev-A", STM32F427IIH6) boards.

Each board runs the **same firmware**. The only difference is a number, `NODE_ID` (1, 2 or 3). Every board:

1. sends a small **heartbeat** frame on the bus at its own period, and
2. prints every frame it receives on its serial port (USART6) and shows a counter from the frame in binary on its 8 LEDs.

Board	CAN ID it sends	Period	Data bytes
node1	`0x101`	500 ms	`01` , counter
node2	`0x102`	600 ms	`02` , counter
node3	`0x103`	700 ms	`03` , counter

Why a simulator?

Renode is an open-source simulator from Antmicro. It runs the exact `.elf` file you would flash onto a board, with the CPU, timers, GPIO, UART and CAN controller emulated. You can run three boards, connect them to one virtual bus and watch every frame without wiring anything or bricking a board.

> **KEEP IN MIND**
>
> A simulator is not the real thing. Renode does not simulate wiring, pin choice, bit timing or bus errors. Chapter 8 lists exactly what changes on real boards. Read it before your first hardware session.

What's in the repository

```
can-bus-onboarding/
├── Makefile                  make · make sim · make test
├── firmware/
│   ├── Core/Src/main.c       ← the file you will edit
│   ├── Core/Src/stm32f4xx_hal_msp.c   pins, clocks and interrupts for CAN1 and USART6
│   ├── Core/Src/stm32f4xx_it.c        interrupt handlers
│   ├── Core/Inc/…            headers, HAL configuration
│   ├── Drivers/              ST's HAL and CMSIS (unchanged)
│   └── STM32F427IIHX_FLASH.ld         linker script
├── sim/
│   ├── dev-a.repl            the simulated Dev-A board
│   ├── can_bus.resc          three boards + one CAN bus
│   └── can_bus.robot         the automatic test
├── docs/                     this guide
└── .devcontainer/            setup for GitHub Codespaces
```

The layout matches a project generated by STM32CubeMX, so what you learn here carries straight over to your team's real code.

2 CAN in ten minutes

2.1 One pair of wires, many boards

A CAN bus is two wires, **CAN_H** and **CAN_L**, usually twisted together. Every board connects to the same two wires, in parallel. A **120 Ω resistor at each end** of the bus stops signals from reflecting back.

The bus only has two states:

- **Dominant = logic 0.** A transmitter pushes CAN_H up and CAN_L down.
- **Recessive = logic 1.** Nobody drives the bus; the resistors pull CAN_H and CAN_L to the same voltage.

If any one board sends a 0, the whole bus reads 0. **“0 wins”** is the most important rule in CAN; it makes arbitration and ACK work (see 2.5 and 2.6).

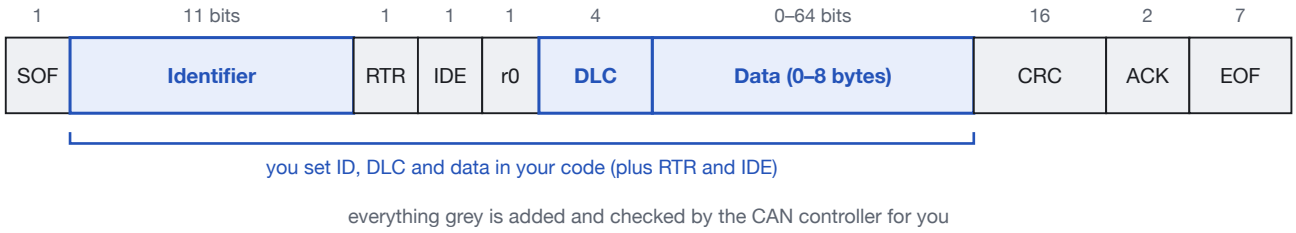
Because the receiver looks at the *difference* between the two wires, noise that hits both wires equally cancels out. That's why CAN survives next to motors.

2.2 Inside each board

- The **CAN controller** lives inside the microcontroller. On the STM32F4 it's called **bxCAN** (there are two: CAN1 and CAN2). It builds frames, handles timing, CRC, ACK and retries, and filters incoming frames. Your code only ever talks to the controller, through the HAL.
- The **transceiver** is a separate chip (an MCP2562 on the Dev-A). It converts the controller's TX/RX logic pins into CAN_H/CAN_L voltages and back.

bxCAN has **3 transmit mailboxes** (frames waiting to go out), **2 receive FIFOs** that hold 3 frames each, and **28 filter banks** shared between CAN1 and CAN2.

2.3 A CAN frame



A standard CAN data frame. At 1 Mbit/s an 8-byte frame takes about 0.13 ms.

Field	Meaning	In the HAL
Identifier	11-bit number (0x000–0x7FF) that says *what* the message is. Also its priority.	`header.StdId`
RTR	Data frame (normal) or remote request (rarely used).	`header.RTR = CAN_RTR_DATA`
IDE	Standard 11-bit ID or extended 29-bit ID.	`header.IDE = CAN_ID_STD`
DLC	Data Length Code: how many data bytes follow (0–8).	`header.DLC`
Data	Up to 8 bytes. What they mean is up to you (or the motor's manual).	`uint8_t data[8]`

2.4 Everyone hears everything

CAN has **no addresses**. When a board sends a frame, *every* other board on the bus receives it. The ID tells receivers what the message is about; each receiver decides whether it cares. The CAN controller can do that sorting in hardware with **filters**, so your code never sees frames it doesn't want. A board never receives its own frames.

2.5 IDs are priorities: arbitration

What if two boards start sending at the same moment? Both send their ID bit by bit, most significant bit first, and both listen to the bus while they send. Because 0 wins, as soon as one board sends a 1 but reads back a 0, it knows someone with a **lower ID** is also sending. It stops and tries again after the bus is free. The winner never notices and no data is lost.

Bit	10	9	8	7	6	5	4	3	2	1	0
node1 sends 0x101	0	0	1	0	0	0	0	0	0	**0**	1
node2 sends 0x102	0	0	1	0	0	0	0	0	0	1 ✗	
bus	0	0	1	0	0	0	0	0	0	**0**	1

The lower the ID, the higher the priority. That's why DJI motor *commands* use ID 0x200 and the motors' *feedback* uses 0x201 and up.

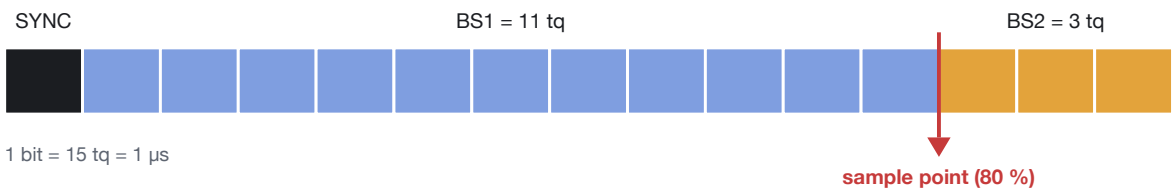
2.6 ACK: somebody has to answer

In the ACK slot of every frame the sender transmits a 1, and every receiver that got the frame correctly overwrites it with a 0. If the sender reads back a 1, nobody received the frame. It reports an error and sends again, forever.

> **COMMON FIRST-DAY BUG**
>
> A board alone on the bus never gets an ACK, so its 3 transmit mailboxes fill up and stay full. Same if the other board isn't running, or uses a different bit rate. The firmware in this project prints `tx skipped: all 3 mailboxes busy` when that happens.

2.7 Bit rate and bit timing

All boards on a bus must use the **same bit rate**. RoboMaster robots use **1 Mbit/s**, the rate DJI motors and ESCs expect. The controller splits each bit into small time slices called **time quanta (tq)** and reads the bus at the **sample point**, near the end of the bit:



The bit timing used in this project: 45 MHz ÷ 3 = 15 MHz, so 1 tq = 66.7 ns; 15 tq = 1 µs per bit = 1 Mbit/s.

You'll see how these numbers are set in code in section 5.3.

3 Set up

You need two tools: the **ARM GCC compiler** (`arm-none-eabi-gcc`) to build the firmware, and **Renode 1.17** to run it. Pick one option.

Option A: GitHub Codespaces (recommended, nothing to install)

A Codespace is a Linux computer in the cloud with VS Code in your browser. The repository tells it what to install.

1. Open the repository on GitHub: github.com/bernininini/can-bus-onboarding (or your team's copy). If you see a 404, ask your lead to give you access.
2. Click the green **Code** button → **Codespaces** tab → **Create codespace on main**.
3. Wait for the setup to finish. The first time takes 3–5 minutes; the terminal at the bottom shows `.devcontainer/setup.sh` installing things. It's done when it prints:

   ```
   arm-none-eabi-gcc (15:13.2.rel1-2) 13.2.1 20231009
   Renode v1.17.0
   ```

4. Click into the terminal and carry on with chapter 4.

> **GOOD TO KNOW**
>
> Personal GitHub accounts get a free monthly Codespaces allowance (enough for many hours on the default 2-core machine). When you're done, stop the codespace from github.com/codespaces; it also stops by itself after 30 minutes of inactivity. Your files are kept.

Option B: your own computer

Linux (Ubuntu 24.04) or Windows (WSL 2 with Ubuntu 24.04)

On Windows, first install WSL: open PowerShell as administrator, run `wsl --install -d Ubuntu-24.04` , restart, and open “Ubuntu” from the Start menu. Then, in the Ubuntu terminal:

```
git clone https://github.com/bernininini/can-bus-onboarding.git
cd can-bus-onboarding
bash .devcontainer/setup.sh       # installs gcc-arm-none-eabi and Renode (asks for your password)
```

macOS (Apple Silicon)

1. Download `renode-1.17.0.osx-arm64-portable.dmg` from github.com/renode/renode/releases/tag/v1.17.0, open it and drag **Renode.app** into Applications.
2. Install the ARM compiler: either `brew install --cask gcc-arm-embedded` , or download `arm-gnu-toolchain-14.3.rel1-darwin-arm64-arm-none-eabi.tar.xz` from developer.arm.com and unpack it.
   The compiler that ships with STM32CubeIDE/CubeCLT is built for Intel Macs and only runs on Apple Silicon if Rosetta is installed.
3. Clone the repository and tell `make` where the tools are (only needed if they're not on your `PATH`):

```
git clone https://github.com/bernininini/can-bus-onboarding.git
cd can-bus-onboarding
export RENODE=/Applications/Renode.app/Contents/MacOS/renode
export PREFIX=$HOME/arm-gnu-toolchain-14.3.rel1-darwin-arm64-arm-none-eabi/bin/arm-none-eabi-
```

MACOS SECURITY PROMPT

If macOS says Renode “can't be opened because Apple cannot check it”, right-click Renode.app → **Open** once, or allow it under System Settings → Privacy & Security.

4 Build and run the simulation

1. **Build the firmware.** From the repository folder:

```
make
```

This compiles the same code three times, once with `-DNODE_ID=1`, once with 2 and once with 3, and ends with:

```
LD  build/node3.elf
   text    data     bss     dec     hex filename
  13172     112    2320   15604    3cf4 build/node3.elf
```

You now have `build/node1.elf`, `node2.elf` and `node3.elf`: one file per board.

2. **Start the simulation.**

```
make sim
```

Renode loads `sim/can_bus.resc`, which creates three Dev-A boards, loads one `.elf` into each, connects all three CAN1 controllers to one virtual bus, and starts them. The first run downloads a small register-description file, so you need internet.

3. **Watch the bus.** After a few seconds you'll see (trimmed a little):

```
node1/usart6: [… virt: 0.1ms …] [node1] up: sending id=0x101 every 500 ms
node2/usart6: [… virt: 0.1ms …] [node2] up: sending id=0x102 every 600 ms
node3/usart6: [… virt: 0.1ms …] [node3] up: sending id=0x103 every 700 ms
node1/usart6: [… virt:  0.5s …] [node1] tx id=0x101 dlc=2 data: 01 00
node2/usart6: [… virt:  0.5s …] [node2] rx id=0x101 dlc=2 data: 01 00
node3/usart6: [… virt:  0.5s …] [node3] rx id=0x101 dlc=2 data: 01 00
node2/usart6: [… virt:  0.6s …] [node2] tx id=0x102 dlc=2 data: 02 00
node1/usart6: [… virt:  0.6s …] [node1] rx id=0x102 dlc=2 data: 02 00
node3/usart6: [… virt:  0.6s …] [node3] rx id=0x102 dlc=2 data: 02 00
node3/usart6: [… virt:  0.7s …] [node3] tx id=0x103 dlc=2 data: 03 00
node1/usart6: [… virt:  0.7s …] [node1] rx id=0x103 dlc=2 data: 03 00
node2/usart6: [… virt:  0.7s …] [node2] rx id=0x103 dlc=2 data: 03 00
node1/usart6: [… virt:    1s …] [node1] tx id=0x101 dlc=2 data: 01 01
```

4. **Stop it.** Type `quit` and press `Enter` (the prompt shows `(node3)`), or press `Ctrl`+`C`.

How to read a log line

```
[19:24:12.4628] [INFO] node2/usart6: [host: 1.47s (+1.15s)|virt: 0.5s (+0.5s)] [node2] rx id=0x101
dlc=2 data: 01 00
```

Part	Meaning
`[19:24:12.4628] [INFO]`	Clock on your computer, and the log level
`node2/usart6:`	Which board, and which UART the text came out of
`host: 1.47s`	Real time that has passed on your computer. The simulator may run slower or faster than real life; that's fine.
`virt: 0.5s`	**Simulated time**, which is what the firmware sees. Use it to check periods: node1 sends at 0.5 s, 1.0 s, 1.5 s… The `(+…)` after each time is the gap since this UART's previous line.
`[node2] rx id=0x101 …`	What the firmware printed with `printf()`

HARMLESS WARNINGS

At startup you'll see a few lines like `WARNING … flash_controller: Unhandled write … ICEN` and `rcc: Unhandled write … RESERVED`. They mean Renode doesn't model the flash cache or one reserved bit. Ignore them.

CHECKPOINT

- Which IDs does node1 receive? Does it ever receive `0x101`? Why not?
- The second data byte goes 00, 01, 02… for each board separately. Where does it come from?
- node3 sends at 0.7 s, 1.4 s, 2.1 s. Which line of `main.c` decides that?

Peek at the LEDs

Renode has no picture of the board, but you can ask for any LED's state. While the simulation runs, type into the Renode prompt:

```
pause
mach set "node1"
gpioPortG.led_a State
gpioPortG.led_b State
start
```

Each `State` line prints `True` (on) or `False` (off). LED_A is bit 0 of the last counter node1 received, LED_B is bit 1, and so on up to LED_H (bit 7).

5 Walk through the code

Open `firmware/Core/Src/main.c`. It's organised like a CubeMX-generated file: CubeMX owns everything outside the `/* USER CODE BEGIN … */` / `/* USER CODE END … */` markers, and your code goes between them. When you regenerate a real project in CubeMX, code between the markers survives and code outside is overwritten.

5.1 One firmware, three boards

```
#ifndef NODE_ID
#define NODE_ID 1
#endif

#define HEARTBEAT_ID        (0x100U + NODE_ID)
#define HEARTBEAT_PERIOD_MS (400U + 100U * NODE_ID)   /* 500, 600, 700 ms */
```

The Makefile passes `-DNODE_ID=1`, `2` or `3` to the compiler, so the same source produces three different boards. Everything else is identical.

5.2 The clock

`SystemClock_Config()` takes the Dev-A's **12 MHz crystal** (HSE) through the PLL up to **180 MHz**, then divides it for the two peripheral buses:

Clock	Setting	Result
PLL	12 MHz ÷ M(6) × N(180) ÷ P(2)	180 MHz system clock
APB1	180 MHz ÷ 4	**45 MHz, which feeds CAN1 and CAN2**
APB2	180 MHz ÷ 2	90 MHz, which feeds USART6

5.3 CAN1 settings and bit timing

```
static void MX_CAN1_Init(void)
{
  hcan1.Instance = CAN1;
  hcan1.Init.Prescaler = 3;
  hcan1.Init.Mode = CAN_MODE_NORMAL;
  hcan1.Init.SyncJumpWidth = CAN_SJW_1TQ;
  hcan1.Init.TimeSeg1 = CAN_BS1_11TQ;
  hcan1.Init.TimeSeg2 = CAN_BS2_3TQ;
  hcan1.Init.TimeTriggeredMode = DISABLE;
  hcan1.Init.AutoBusOff = ENABLE;
  hcan1.Init.AutoWakeUp = DISABLE;
  hcan1.Init.AutoRetransmission = ENABLE;
  hcan1.Init.ReceiveFifoLocked = DISABLE;
  hcan1.Init.TransmitFifoPriority = DISABLE;
  if (HAL_CAN_Init(&hcan1) != HAL_OK)
  {
    Error_Handler();
  }
}
```

The bit rate is set by three numbers:

```
bit rate = APB1 clock / Prescaler / (1 + BS1 + BS2)
         = 45 MHz     / 3         / (1 + 11 + 3)    = 1 000 000 bit/s
sample point = (1 + BS1) / (1 + BS1 + BS2) = 12 / 15 = 80 %
```

Setting	What it does
`Mode`	`NORMAL` talks to the bus. `LOOPBACK` sends frames to itself (handy for testing one board alone).
`AutoBusOff`	If too many errors happen the controller goes “bus-off” to protect the bus. `ENABLE` lets it recover on its own.
`AutoRetransmission`	Resend a frame that wasn't acknowledged, or lost arbitration. Leave it on.
`TransmitFifoPriority`	`DISABLE` : when several mailboxes are full, the lowest ID goes first. `ENABLE` : oldest first.

IF YOU CHANGE THE CLOCK

If APB1 is not 45 MHz, the bit rate is wrong and real boards stop talking to each other or to the motors. In CubeMX, the CAN1 parameter page shows the resulting “Baud Rate”; always check it says 1000000.

5.4 Pins: `stm32f4xx_hal_msp.c`

`HAL_CAN_Init()` calls `HAL_CAN_MspInit()` , which turns on the clocks, sets up the pins and enables the interrupt:

```
/* CAN1 on the Dev-A: PD0 = CAN1_RX, PD1 = CAN1_TX (from the schematic,
 * NOT the CubeMX default of PA11/PA12). RX gets a pull-up. */
GPIO_InitStruct.Pin = GPIO_PIN_0;
GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
GPIO_InitStruct.Pull = GPIO_PULLUP;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
GPIO_InitStruct.Alternate = GPIO_AF9_CAN1;
HAL_GPIO_Init(GPIOD, &GPIO_InitStruct);
…
HAL_NVIC_SetPriority(CAN1_RX0_IRQn, 5, 0);
HAL_NVIC_EnableIRQ(CAN1_RX0_IRQn);
```

The STM32 can route CAN1 to several different pins; the board designer picked PD0/PD1 and wired them to the transceiver. You find that in the **schematic**, not in CubeMX.

5.5 The filter: which frames do we keep?

```
static void CAN_Filter_Config(void)
{
  CAN_FilterTypeDef filter = {0};
  filter.FilterBank = 0;
  filter.FilterMode = CAN_FILTERMODE_IDMASK;
  filter.FilterScale = CAN_FILTERSCALE_32BIT;
  filter.FilterIdHigh = 0x0000;
  filter.FilterIdLow = 0x0000;
  filter.FilterMaskIdHigh = 0x0000;
  filter.FilterMaskIdLow = 0x0000;
  filter.FilterFIFOAssignment = CAN_FILTER_FIFO0;
  filter.FilterActivation = ENABLE;
  filter.SlaveStartFilterBank = 14;   /* banks 0–13 for CAN1, 14–27 for CAN2 */
  …
}
```

Until at least one filter is active, bxCAN **throws every frame away**, so you always need this. In **mask mode**, each mask bit says whether that ID bit matters:

- mask bit **1** = “the received ID bit must equal the filter ID bit”
- mask bit **0** = “don't care”

An all-zero mask means “don't care about anything”, so every frame is accepted into FIFO0. In exercise 3 you'll change this to accept only one ID.

5.6 Start the controller

```
CAN_Filter_Config();
if (HAL_CAN_Start(&hcan1) != HAL_OK) { Error_Handler(); }
/* Call HAL_CAN_RxFifo0MsgPendingCallback() whenever a frame arrives */
if (HAL_CAN_ActivateNotification(&hcan1, CAN_IT_RX_FIFO0_MSG_PENDING) != HAL_OK) {
Error_Handler(); }
```

`HAL_CAN_Init()` leaves the controller in *initialization mode*, off the bus. `HAL_CAN_Start()` joins the bus. `HAL_CAN_ActivateNotification()` turns on the “frame received” interrupt.

5.7 Sending a frame

```
static void send_heartbeat(uint8_t counter)
{
  CAN_TxHeaderTypeDef header = {0};
  header.StdId = HEARTBEAT_ID;     /* 11-bit identifier */
  header.IDE = CAN_ID_STD;         /* standard (11-bit), not extended (29-bit) */
  header.RTR = CAN_RTR_DATA;       /* a data frame, not a remote request */
  header.DLC = 2;                  /* number of data bytes, 0..8 */
  header.TransmitGlobalTime = DISABLE;

  uint8_t data[2] = { NODE_ID, counter };
  uint32_t mailbox;

  if (HAL_CAN_GetTxMailboxesFreeLevel(&hcan1) == 0)
  {
    printf("[node%d] tx skipped: all 3 mailboxes busy\r\n", NODE_ID);
    return;
  }
  if (HAL_CAN_AddTxMessage(&hcan1, &header, data, &mailbox) != HAL_OK) { … }
  …
}
```

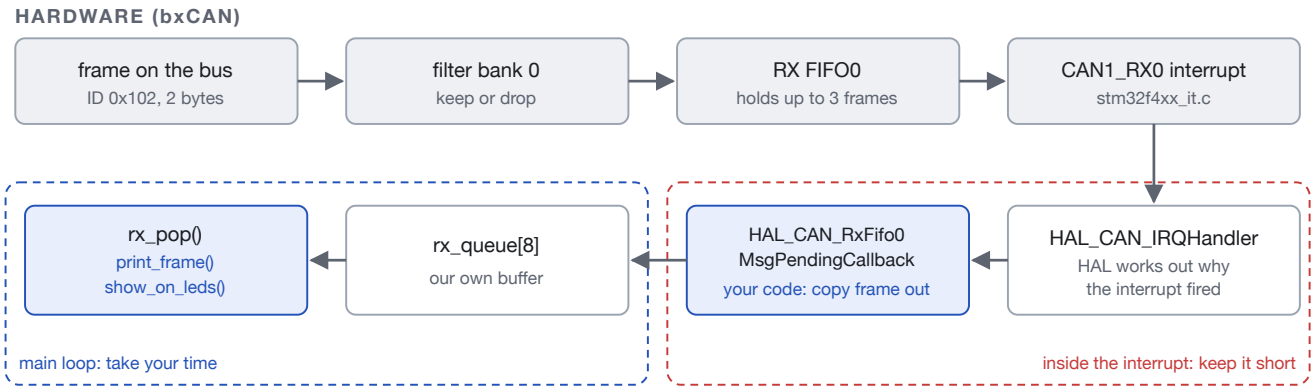
`HAL_CAN_AddTxMessage()` doesn't wait. It copies the frame into a free mailbox and returns; the controller sends it as soon as the bus is free and retries on its own if needed.

The main loop calls `send_heartbeat()` on a timer built from `HAL_GetTick()`, which counts milliseconds since boot (SysTick interrupt):

```
if ((int32_t)(HAL_GetTick() - next_tx) >= 0)
{
  next_tx += HEARTBEAT_PERIOD_MS;
  send_heartbeat(counter++);
}
```

Writing it as a subtraction keeps working when the 32-bit millisecond counter wraps around after 49 days.

5.8 Receiving a frame



The path of a received frame. Hardware does the left-to-right top row; your code handles the bottom row.

When a frame passes the filter and lands in FIFO0, the CPU jumps to `CAN1_RX0_IRQHandler()`, which calls the HAL, which calls **your callback**:

```
void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *hcan)
{
  RxFrame *slot = &rx_queue[rx_head];
  if (HAL_CAN_GetRxMessage(hcan, CAN_RX_FIFO0, &slot->header, slot->data) != HAL_OK)
  {
    return;
  }

  uint32_t next = (rx_head + 1) % RX_QUEUE_LEN;
  if (next == rx_tail)
  {
    rx_dropped++;   /* queue full: this frame is lost */
    return;
  }
  rx_head = next;
}
```

The callback only copies the frame out of the hardware FIFO into our own queue. It does **not** print. Printing 40 characters at 115200 baud takes about 3.5 ms, and while an interrupt runs, other interrupts of the same or lower priority wait. On a robot that could mean missed motor feedback. Slow work belongs in the main loop:

```
RxFrame f;
while (rx_pop(&f))
{
  print_frame(&f);
  if (f.header.DLC >= 2)
  {
    show_on_leds(f.data[1]);
  }
}
```

`rx_pop()` switches interrupts off for a few instructions (`__disable_irq()` … `__enable_irq()`) so the callback can't change the queue halfway through a read. That short protected stretch is called a **critical section**.

5.9 LEDs

```
/* LED_A shows bit 0, LED_H shows bit 7. The LEDs are wired active-low:
 * writing 0 (RESET) to the pin turns the LED on. */
static void show_on_leds(uint8_t value)
{
  for (int bit = 0; bit < 8; bit++)
  {
    GPIO_PinState state = ((value >> bit) & 1U) ? GPIO_PIN_RESET : GPIO_PIN_SET;
    HAL_GPIO_WritePin(LED_GPIO_Port, LED_A_Pin << bit, state);
  }
}
```

On the Dev-A, each LED sits between 3.3 V (through a 1 kΩ resistor) and its pin, PG1–PG8. Current only flows, and the LED only lights, when the pin is **low**. That's called **active-low**, and it's why `MX_GPIO_Init()` sets the pins high first: all LEDs start off.

5.10 printf

`printf()` doesn't know about UARTs. The C library calls `_write()` to output characters, and `main.c` provides one that sends them out of USART6:

```
int _write(int file, char *ptr, int len)
{
  (void)file;
  HAL_UART_Transmit(&huart6, (uint8_t *)ptr, (uint16_t)len, HAL_MAX_DELAY);
  return len;
}
```

In the simulator, Renode prints whatever arrives on USART6 into its log; on a real board you'd read it with a USB-to-serial adapter (chapter 8).

6 Exercises

Do them in order. After every change: save, run `make sim` (it rebuilds automatically), compare with the expected output, then `quit`. Solutions are in Appendix A; try first.

> **UNDO EVERYTHING**
>
> To get the original `main.c` back at any time: `git checkout firmware/Core/Src/main.c`. To keep your work but start fresh: `git stash`.

Exercise 1: Read the bus (no code)

Run `make sim` and answer, using only the log:

1. At 1.4 s simulated time, which boards send and which receive?
2. How many frames does node2 receive in the first 2 seconds? Predict first, then count.
3. What is the last counter node1 has received at 1.85 s? Pause there and check its LEDs with `gpioPortG.led_a State` etc. Do they match the binary value?

Exercise 2: Send a 16-bit number

Goal: each heartbeat carries 4 bytes. Bytes 2 and 3 hold a 16-bit value, `NODE_ID × 1000 + counter`, **high byte first** ("big-endian", the order DJI motors use). Receivers print the decoded value.

1. In `send_heartbeat()`: set `DLC` to 4 and build a 4-byte `data[]`. Use `value >> 8` for the high byte and `value & 0xFF` for the low byte.
2. In `print_frame()`: if `DLC >= 4`, rebuild the value with `(data[2] << 8) | data[3]` and print it.

Expected:

```
[node1] tx id=0x101 dlc=4 value=1000
[node2] rx id=0x101 dlc=4 data: 01 00 03 E8  -> value=1000
[node3] rx id=0x101 dlc=4 data: 01 00 03 E8  -> value=1000
[node2] tx id=0x102 dlc=4 value=2000
[node1] rx id=0x102 dlc=4 data: 02 00 07 D0  -> value=2000
```

Think about it: 1000 is `0x03E8`. What would the receiver print if it got the byte order backwards?

Exercise 3: Filters

Goal: every board accepts **only ID 0x101** (node1's heartbeat). Change only `CAN_Filter_Config()`.

In 32-bit mask mode, the 11-bit standard ID sits in the **top 11 bits** of the 32-bit filter register, so inside the 16-bit `FilterIdHigh` / `FilterMaskIdHigh` fields it's shifted left by 5:

Register bits	31 … 21	20 … 3	2	1	0
Meaning	STID[10:0], the standard ID	EXID (extended IDs only)	IDE	RTR	0
HAL field	`FilterIdHigh` holds bits 31…16 and `FilterIdLow` holds bits 15…0, so the standard ID lands in bits 15…5 of `FilterIdHigh`.				

1. Set the filter ID to 0x101 (remember the shift).
2. Set the mask so all 11 ID bits must match: 0x7FF, shifted the same way.

Expected: node1 receives **nothing** (it only hears others, and only 0x101 is allowed). node2 and node3 receive only 0x101:

```
[node1] tx id=0x101 dlc=2 data: 01 00
[node2] rx id=0x101 dlc=2 data: 01 00
[node3] rx id=0x101 dlc=2 data: 01 00
[node2] tx id=0x102 dlc=2 data: 02 00
[node3] tx id=0x103 dlc=2 data: 03 00
[node1] tx id=0x101 dlc=2 data: 01 01
```

Think about it: which mask accepts *both* 0x102 and 0x103 but nothing else? (Hint: they differ only in bit 0.)

Exercise 4: A DJI-style motor command

DJI's C620 motor controllers listen for **ID 0x200**: 8 bytes holding four signed 16-bit current commands, for motors 1 to 4, each high byte first (Appendix B). Let's fake it.

Goal:

- node1 is the “main controller”. Right after each heartbeat it also sends ID 0x200 with the values `1000, −1000, 0, 0`.
- node2 pretends to be motor 1 (bytes 0–1) and node3 pretends to be motor 2 (bytes 2–3). Each decodes *its own* value and prints it.

1. Write `send_motor_command(int16_t m1, int16_t m2, int16_t m3, int16_t m4)` next to `send_heartbeat()`, and add its prototype in the `USER CODE PFP` block.
2. In the main loop, call it after `send_heartbeat()`, but only when `NODE_ID == 1`.
3. In the receive loop: if the ID is 0x200 and this isn't node1, pick bytes `2 × (NODE_ID − 2)` and the one after, combine them into an `int16_t`, and print it.

Expected:

```
[node2] rx id=0x200 dlc=8 data: 03 E8 FC 18 00 00 00 00
[node3] rx id=0x200 dlc=8 data: 03 E8 FC 18 00 00 00 00
[node2] my current command = 1000
[node3] my current command = -1000
```

Think about it: −1000 arrives as `FC 18`. Why? (Look up “two's complement”.) And why do the LEDs suddenly show `11101000` after a 0x200 frame?

7 Check your work automatically

The repository has an automatic test in `sim/can_bus.robot`, written for Renode's Robot Framework integration. It starts the three boards and checks the serial output and LEDs for you:

Test	Passes when…
Every Node Hears The Other Two	each board prints `rx` lines for the other two IDs
No Node Hears Itself	no board prints an `rx` line for its own ID
LEDs Show The Received Counter In Binary	after node2 receives counter 1, LED_A is on and LED_B is off

```
make test
```

When it finishes, each test is reported as `OK` and the run ends with `Tests finished successfully :)`. Details are written to `log.html`, which you can open in a browser.

EXPECTED FAILURES

The test checks the *original* behaviour. After exercise 3 (filters), “Every Node Hears The Other Two” fails, and it should: the boards really don't hear each other any more.

ON A MAC

`renode-test` needs a few Python packages the first time: `python3 -m pip install -r /Applications/Renode.app/Contents/MacOS/tests/requirements.txt`, then `make test RENODE_TEST=/Applications/Renode.app/Contents/MacOS/renode-test`. Codespaces and `setup.sh` already handle this.

The same test also runs on GitHub on every push (`.github/workflows/ci.yml`), so a green check mark on the repository means the project builds and works.

8 From the simulator to real boards

8.1 What the simulator does not check

Renode models CAN at the level of whole frames. Several things that break real buses are invisible in the simulator:

On real hardware	In Renode
CAN1 must be on **PD0/PD1** (the pins wired to the transceiver)	Pin choice is ignored. PA11/PA12 would “work” too.
`CAN1_RX` needs a pull-up	Not modeled
All boards must use the **same bit rate**	Bit timing is stored but ignored. A board at 500 kbit/s still “works”.
Someone must **ACK**; errors, retries and bus-off happen	Every frame succeeds instantly, even with no other board.
Arbitration takes time; a busy bus delays frames	Frames are delivered instantly, no contention.
Wiring, termination, loose connectors, noise	Not modeled

The repository also works around one Renode quirk: ST's `HAL_CAN_Init()` requests initialization mode while the controller is still asleep, which real bxCAN allows and Renode's model didn't. A one-line hook in `sim/dev-a.repl` fixes that. You don't need it on hardware.

8.2 What you need

- 2 or 3 Dev-A boards, powered the way your team normally does it
- CAN cables between the boards' **CAN1** ports (CAN_H to CAN_H, CAN_L to CAN_L)
- An ST-Link (SWD) to flash, and a 3.3 V USB-to-serial adapter to read USART6

8.3 Wiring checks

- **Termination.** With everything powered off, measure the resistance between CAN_H and CAN_L. About **60 Ω** (two 120 Ω resistors in parallel) is right. The Dev-A schematic shows a 120 Ω resistor next to each MCP2562 transceiver, so check how many are actually connected on your bus.
- **Serial.** USART6 is TX = PG14, RX = PG9 on the MCU. Find which connector they go to in the Dev-A schematic. Connect adapter RX to board TX, adapter TX to board RX, and GND to GND. Open the port at **115200 baud, 8N1**.

8.4 Flash each board

Each board gets its own file. With STM32CubeProgrammer's command-line tool:

```
STM32_Programmer_CLI -c port=SWD -w build/node1.elf -v -rst     # first board
STM32_Programmer_CLI -c port=SWD -w build/node2.elf -v -rst     # second board
```

You should see the same `tx` / `rx` lines as in the simulator, and the LEDs counting in binary.

8.5 Rebuild this project in CubeMX

Your team's projects start in STM32CubeMX. To recreate this one (good practice!), create a new project for **STM32F427IIHx** and set:

Where in CubeMX	Setting
System Core → RCC	High Speed Clock (HSE): Crystal/Ceramic Resonator
Clock Configuration	Input 12 MHz, PLL source HSE, /M = 6, ×N = 180, /P = 2, SYSCLK = PLLCLK (180 MHz), APB1 ÷4 (45 MHz), APB2 ÷2 (90 MHz)
Connectivity → CAN1	Activated. Move the pins to **PD0** (CAN1_RX) and **PD1** (CAN1_TX). GPIO Settings: PD0 pull-up. Parameters: Prescaler 3, Time Quanta in Bit Segment 1 = 11, Bit Segment 2 = 3, ReSynchronization Jump Width = 1; check that Baud Rate shows 1000000. NVIC Settings: enable “CAN1 RX0 interrupt”.
Connectivity → USART6	Asynchronous, 115200 baud. Pins **PG14** (TX) and **PG9** (RX).
Pinout view	PG1–PG8: GPIO_Output, labels LED_A…LED_H. GPIO settings: output level **High** (LEDs off).
Project Manager	Toolchain: CMake or Makefile. Generate code.

Then copy everything between the `USER CODE` markers from this repository's `main.c` into the generated `main.c`.

8.6 First-time hardware problems

Symptom	Likely cause
`tx skipped: all 3 mailboxes busy`	Nobody ACKs: the other board isn't running, isn't wired to CAN1, uses a different bit rate, or wrong pins in CubeMX.
Nothing at all on the serial port	TX/RX swapped, missing GND, wrong baud rate, or stuck in `Error_Handler()` (check with the debugger).
Works for a while, then stops	Bus-off from errors: check termination and connectors.

9 Troubleshooting

Message	Fix
`arm-none-eabi-gcc: No such file or directory`	The compiler isn't on your `PATH`. Pass it: `make PREFIX=/path/to/bin/arm-none-eabi-`.
`bad CPU type in executable` (Mac)	You're using an Intel-only compiler (e.g. from STM32CubeIDE). Use the arm64 ARM GNU toolchain (chapter 3).
`renode: command not found`	`make sim RENODE=/path/to/renode`
Renode can't download `STM32F40x.svd.gz`	The first run needs internet. Try again on a working connection.
Only the three `up:` lines, then nothing	The firmware is stuck, usually in `Error_Handler()`. In the Renode prompt: `pause`, `mach set "node1"`, `cpu PC`, then look up the address with `arm-none-eabi-addr2line -f -e build/node1.elf 0x…`
No `rx` lines but `tx` lines appear	Your filter rejects everything, or you removed `HAL_CAN_ActivateNotification()`.
Compile error after an exercise	Read the *first* error line; later ones are often caused by it. Missing prototype? Missing semicolon?

A Solutions

These were all built and run in the simulator; the expected output in chapter 6 comes from them.

Exercise 2

In `send_heartbeat()`:

```
header.DLC = 4;
…
uint16_t value = NODE_ID * 1000 + counter;
uint8_t data[4] = { NODE_ID, counter, value >> 8, value & 0xFF };
…
printf("[node%d] tx id=0x%03lX dlc=%lu value=%u\r\n",
       NODE_ID, header.StdId, header.DLC, value);
```

In `print_frame()`, just before the final `printf("\r\n")`:

```
if (f->header.DLC >= 4)
{
  uint16_t value = (f->data[2] << 8) | f->data[3];
  printf("  -> value=%u", value);
}
```

Backwards byte order would turn 1000 (0x03E8) into 0xE803 = 59395.

Exercise 3

```
filter.FilterIdHigh = 0x101 << 5;
filter.FilterIdLow = 0x0000;
filter.FilterMaskIdHigh = 0x7FF << 5;
filter.FilterMaskIdLow = 0x0000;
```

To accept 0x102 and 0x103: ID `0x102 << 5`, mask `0x7FE << 5` (bit 0 is “don't care”).

Exercise 4

Prototype, in `USER CODE BEGIN PFP`:

```
static void send_motor_command(int16_t m1, int16_t m2, int16_t m3, int16_t m4);
```

The function, in `USER CODE BEGIN 4`:

```
static void send_motor_command(int16_t m1, int16_t m2, int16_t m3, int16_t m4)
{
  CAN_TxHeaderTypeDef header = {0};
  header.StdId = 0x200;
  header.IDE = CAN_ID_STD;
  header.RTR = CAN_RTR_DATA;
  header.DLC = 8;
  uint8_t data[8] = { m1 >> 8, m1, m2 >> 8, m2, m3 >> 8, m3, m4 >> 8, m4 };
  uint32_t mailbox;
  HAL_CAN_AddTxMessage(&hcan1, &header, data, &mailbox);
}
```

The main loop, in `USER CODE BEGIN 3`:

```
if ((int32_t)(HAL_GetTick() - next_tx) >= 0)
{
  next_tx += HEARTBEAT_PERIOD_MS;
  send_heartbeat(counter++);
  if (NODE_ID == 1)
  {
    send_motor_command(1000, -1000, 0, 0);
  }
}

RxFrame f;
while (rx_pop(&f))
{
  print_frame(&f);
  if (f.header.StdId == 0x200 && NODE_ID != 1)
  {
    int slot = NODE_ID - 2;   /* node2 = motor 1 = bytes 0-1 */
    int16_t current = (int16_t)((f.data[2 * slot] << 8) | f.data[2 * slot + 1]);
    printf("[node%d] my current command = %d\r\n", NODE_ID, current);
  }
  if (f.header.DLC >= 2)
  {
    show_on_leds(f.data[1]);
  }
}
```

−1000 in 16-bit two's complement is 65536 − 1000 = 64536 = 0xFC18. The LEDs show `data[1]` of *every* frame, and in a 0x200 frame that's the low byte of motor 1's command, 0xE8 = 11101000.

B DJI motor CAN frames

This is what exercise 4 was preparing you for. All values are signed 16-bit, **high byte first**, at 1 Mbit/s. Always check the official manual for your motor and ESC (e.g. the C620 and GM6020 user guides); this table is a summary.

C620 / C610 ESCs (M3508, M2006 motors)

Direction	ID	Data
Command (you send)	`0x200`	bytes 0–1: motor 1 · 2–3: motor 2 · 4–5: motor 3 · 6–7: motor 4. C620 current command −16384…16384 (about −20…20 A).
Command (you send)	`0x1FF`	the same layout for motors 5–8
Feedback (motor sends)	`0x200 + motor ID` (0x201–0x208)	bytes 0–1: rotor angle 0…8191 (= 0…360°) · 2–3: speed in rpm · 4–5: actual torque current · 6: temperature °C (C620) · 7: unused

GM6020 gimbal motor

Direction	ID	Data
Command	`0x1FF` / `0x2FF`	four voltage commands (−30000…30000) for motor IDs 1–4 / 5–7
Feedback	`0x204 + motor ID`	same layout as the C620 feedback

WATCH OUT

Command ID `0x1FF` is shared: C620 motors 5–8 and GM6020 motors 1–4 both listen to it. Two devices must never decode the same bytes on the same bus, so plan your motor IDs.

Reading feedback is exactly the decoding you did in exercises 2 and 4: check the ID, then combine two bytes into an `int16_t`.

C Renode cheat sheet

Type these at the Renode prompt while `make sim` is running.

Command	What it does
`pause` / `start`	Freeze / resume all three boards
`quit`	Exit Renode
`mach set "node2"`	Choose which board the next commands talk to
`gpioPortG.led_a State`	Is LED_A on (`True`) or off (`False`)?
`cpu PC`	Where is this board's CPU right now?
`sysbus ReadDoubleWord 0x4000641C`	Read a register directly (this one is `CAN1_BTR`)
`emulation RunFor "00:00:02"`	Run for exactly 2 s of simulated time, then pause
`machine Reset`	Reset the selected board (like pressing its reset button)
`help`	List everything

How the simulation is put together

`sim/dev-a.repl` describes one board: it starts from Renode's generic STM32F4 file and adds the parts this project uses (USART6, the 8 LEDs, the 180 MHz SysTick, the CAN init fix). `sim/can_bus.resc` builds the bus:

```
emulation CreateCANHub "canHub"

mach create "node1"
machine LoadPlatformDescription $ORIGIN/dev-a.repl
sysbus LoadELF $ORIGIN/../build/node1.elf
connector Connect sysbus.can1 canHub
showAnalyzer sysbus.usart6 Antmicro.Renode.Analyzers.LoggingUartAnalyzer
… (the same for node2 and node3)
```

Want a fourth board? Copy one block, call it `node4` , add `4` to `NODES` in the Makefile, and run `make sim` .

Further reading

- STM32F427 reference manual (RM0090), chapter “Controller area network (bxCAN)”
- ST's HAL manual UM1725, section “HAL CAN Generic Driver”
- Renode documentation: renode.readthedocs.io
- Dev-A schematic and user manual from your team's resources